

Министерство науки и высшего
образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3
по курсу «ЛиОАвИЗ»
на тему «Динамические структуры данных
(списки, стек, очередь, приоритетная очередь)»

Выполнил:

студент группы 25BBB1

Садыков Д.Р.

Приняли:

к.т.н., доцент Юрова О.В.,

ст. преподаватель Деев М.В.

1. Цель работы

Изучение принципов организации и функционирования динамических структур данных (односвязные линейные списки, стек, очередь, приоритетная очередь). Приобретение практических навыков динамического управления оперативной памятью в языке программирования Си с использованием функций `malloc()` и `free()`, разработки алгоритмов добавления, извлечения, сквозного просмотра, поиска и удаления элементов в связанных структурах данных.

2. Лабораторное задание

В соответствии с методическими указаниями к лабораторной работе № 3 требуется выполнить следующие задания:

- Изучить базовую реализацию односвязного линейного списка, содержащего текстовые наименования объектов (`inf[256]`), функции добавления (`spstore`), просмотра (`review`), поиска (`find`) и удаления (`del`).
- На основе базовой связной структуры реализовать структуру данных Очередь (`Queue`), работающую по принципу FIFO (`First In, First Out` — первым пришел, первым обслужен), с операциями постановки в хвост и извлечения из головы.
- На основе приведенного связного списка реализовать структуру данных Стек (`Stack`), функционирующую по принципу LIFO (`Last In, First Out` — последним пришел, первым обслужен), с операциями помещения в вершину (`push`) и извлечения из вершины (`pop`).
- Реализовать структуру данных Приоритетная очередь (`Priority Queue`) путем упорядоченного включения элементов в список в соответствии с числовым приоритетом объекта (элемент с более высоким приоритетом размещается перед элементом с меньшим приоритетом, а при равных приоритетах сохраняется естественный порядок поступления FIFO).
- Реализовать функции поиска и выборочного удаления элементов по наименованию в приоритетной очереди, а также функции полной очистки выделенной динамической памяти.
- Разработать демонстрационный модуль и интерактивное меню пользователя на языке Си, провести тестирование всех реализованных структур и составить подробный отчет о результатах работы.

3. Теоретические сведения

3.1. Особенности динамических структур данных и управление памятью

В отличие от статических структур данных (таких как стандартные одномерные и многомерные массивы), размер которых фиксируется на этапе компиляции и не может изменяться в процессе выполнения программы, динамические структуры данных обладают следующими ключевыми свойствами:

- Динамическое выделение памяти: память под каждый элемент (узел) выделяется из свободной области оперативной памяти (кучи, heap) непосредственно в момент создания элемента по мере необходимости.
- Связность через указатели: узлы структуры не обязаны располагаться в смежных ячейках оперативной памяти; логическая связь между элементами обеспечивается явным хранением адреса следующего узла в поле-указателе.
- Отсутствие жесткого ограничения на размер: структура может неограниченно расти вплоть до полного исчерпания физической оперативной памяти вычислительной системы и легко уменьшаться при освобождении узлов.

3.2. Принцип функционирования Стека (LIFO)

Стек (Stack) — динамическая структура данных, работающая по принципу LIFO (Last In, First Out — «последним пришел, первым обслужен»). Доступ к элементам осуществляется исключительно через один конец структуры, называемый вершиной стека (TOP). Операция push (проталкивание) связывает новый элемент с текущей вершиной. Операция pop (выталкивание) извлекает информационное поле из вершины и переносит указатель на следующий элемент.

3.3. Принцип функционирования Очереди (FIFO)

Очередь (Queue) — структура данных, организованная по дисциплине обслуживания FIFO (First In, First Out — «первым пришел, первым ушел»). Для эффективного функционирования используются два независимых указателя: Указатель на начало (HEAD) для извлечения элементов и Указатель на конец (TAIL) для добавления элементов в хвост. Использование двух указателей обеспечивает константную временную сложность $O(1)$.

3.4. Принцип функционирования Приоритетной очереди

Приоритетная очередь (Priority Queue) — структура данных, в которой каждый элемент помимо данных несет характеристику важности (приоритет - priority). Обслуживание элементов производится строго в порядке убывания их приоритета. При поступлении нового элемента список просматривается от головы; элемент размещается перед первым встреченным узлом, имеющим строго меньший приоритет. Если в очереди присутствуют элементы с таким же приоритетом, новый элемент помещается за ними, что гарантирует сохранение стабильности очереди (FIFO).



Рисунок 1 - Схемы организации структур данных.

4. Описание программной реализации

Программный комплекс разработан на стандартном языке Си (C99/C11) и скомпилирован с использованием компилятора GCC в среде Linux (WSL). Программа состоит из независимых модулей для работы со стеком, очередью и приоритетной очередью.

Структура данных	Принцип доступа	Сложность вставки	Сложность извлечения	Точка добавления	Точка извлечения
Стек (Stack)	LIFO	O(1)	O(1)	Вершина (TOP)	Вершина (TOP)
Очередь (Queue)	FIFO	O(1)	O(1)	Хвост (TAIL)	Голова (HEAD)
Приоритетная очередь	По приоритету	O(N)	O(1)	По позиции приоритета	Голова (HEAD)
Линейный список	Произвольный	O(1) в конец	O(N) по ключу	Конец (last)	По значению (del)

5. Анализ результатов тестирования

- **Корректность работы Стекa:** при добавлении документов 1, 2, 3 последним был добавлен «Документ_3». При выполнении операции pop() именно он был первым извлечен из вершины, что полностью подтверждает дисциплину LIFO.
- **Корректность работы Очереди:** элементы «Клиент_А», «Клиент_Б», «Клиент_В» добавлялись в хвост очереди. При вызове dequeue() первым был обслужен «Клиент_А», первым вставший в очередь, что доказывает соблюдение принципа FIFO.
- **Корректность работы Приоритетной очереди:** при добавлении элементов с различными приоритетами (от 1 до 10) структура автоматически выстроила элементы строго по убыванию приоритета. «Крит_ошибка» (приоритет 10) занял первое место в очереди, опередив ранее добавленные элементы с меньшим приоритетом.

- **Сохранение очередности при равных приоритетах:** элементы «БД_запрос1» и «БД_запрос2» имеют одинаковый приоритет 5. В очереди «БД_запрос1» остался перед «БД_запрос2», сохранив хронологический порядок добавления (свойство стабильности).
- **Поиск и удаление:** элемент «Срочно» был успешно найден по наименованию и корректно удален из середины связного списка с восстановлением целостности указателей.

6. Выводы

- Изучены теоретические основы организации динамических структур данных на базе связных списков и указателей в языке программирования Си.
- Освоены практические навыки динамического выделения (malloc) и своевременного освобождения (free) оперативной памяти, исключая утечки памяти.
- На основе базового односвязного списка успешно реализованы классические структуры данных Стек (LIFO) и Очередь (FIFO).
- Реализована Приоритетная очередь с автоматической упорядоченной вставкой элементов по убыванию приоритета и сохранением хронологического порядка (FIFO) для элементов с одинаковым приоритетом.
- Реализованы функции поиска и удаления произвольных узлов списка по строковому наименованию, а также полное освобождение динамических структур при завершении программы.
- Проведено всестороннее тестирование разработанных алгоритмов в терминале WSL, полностью подтвердившее корректность их функционирования.